CSE201 Homework 3

Minimum Spanning Trees and Shortest Paths

Total: 100 points

As before, Problem 1 will appear in every version of Quiz 3. The other quiz problem will be a variant of one of the remaining homework problems.

Describe your algorithms precisely and explain why they are correct. Clear explanations in words are welcome. For a counterexample, show the proposed algorithm's result and a better feasible solution. **You do not have to explain your running time explicitly in any problem**, but your algorithm must meet the stated running-time target.

Use adjacency lists. Unless a problem says otherwise, you may use the algorithms and results from lecture without proving them again. Use the lecture notation $v.d$ for a distance label and $v.\pi$ for a predecessor. Count each arithmetic operation and comparison as $O(1)$ time.

Problem 1. Safe edges and the MST **20 points**

Let $G = (V, E)$ be a connected, undirected graph whose edge weights are all distinct. You may assume that G has a unique minimum spanning tree T.

As in lecture, an edge e is *safe* if it is the lightest edge crossing some cut $(S, V \setminus S)$, where $\varnothing \neq S \subsetneq V$. An edge crosses this cut if exactly one endpoint is in S.

Show that

$$e \text{ is safe} \quad \Longleftrightarrow \quad e \in T.$$

For this problem, give the exchange argument for each direction; do not simply cite the safe-edge theorem.

(a) Show that every safe edge belongs to T.

(b) Show that every edge of T is safe.

Problem 2. Updating an MST **20 points**

Let $G = (V, E)$ be a connected, undirected graph with distinct edge weights, and suppose its MST T is already given. Consider the following two changes **separately**, each starting from the original G and T.

For each part, describe an $O(|V| + |E|)$ algorithm and explain why it returns an MST of the changed graph. Do not recompute an MST from scratch using Prim's or Kruskal's algorithm.

(a) Add a new edge $e = (u, v) \notin E$ between two distinct vertices, with a weight different from every existing edge weight. How can you update T?

(b) Delete an edge e that belongs to T. How can you update T, or report that the changed graph has no spanning tree?

Problem 3. Bellman–Ford on small components **20 points**

Let $G = (V, E)$ be a weighted directed graph with source s, and let $n = |V|$ and $m = |E|$. Edge weights may be negative, but assume that G has no negative-weight cycles. Every strongly connected component has at most Δ vertices, where $\Delta \geq 1$.

Compute the shortest-path distance from s to every vertex in $O(n + \Delta m)$ time. Return ∞ for unreachable vertices. You may use the SCC algorithm from lecture without proving it again.

(a) Describe your algorithm, including how you initialize and update the distance labels.

(b) Explain why your algorithm returns the correct distances.

Hint: Process the strongly connected components in a suitable order, and use Bellman–Ford within each component.

Problem 4. One free edge **20 points**

Let $G = (V, E)$ be a directed graph with nonnegative edge weights and two distinct vertices s, t. You have a coupon that lets you traverse **at most one edge for free**. Every other edge costs its original weight. Find the minimum cost of traveling from s to t with this coupon. Return ∞ if t is unreachable from s.

(a) Consider this proposed method: first find a shortest s–t path using the original weights, then use the coupon on a heaviest edge of that path. Give a counterexample showing that this method can fail. Your example must have a unique shortest s–t path under the original weights.

(b) Design a correct algorithm that runs in $O((|V| + |E|) \log |V|)$ time. Explain why it finds the best route, including the best choice of where to use the coupon.

Problem 5. Currency exchange with fees **20 points**

There are $n \geq 2$ currencies. For each allowed exchange $(i, j) \in E$, one unit of currency i can be exchanged for $r_{ij} > 0$ units of currency j before fees. Each exchange charges a fixed fraction f, where $0 \leq f < 1$, of the amount received. Thus exchanging x units of currency i gives

$$x(1 - f)r_{ij}$$

units of currency j. Rates and the fee fraction stay fixed throughout all exchanges. Amounts may be divided arbitrarily.

An *arbitrage opportunity* is a sequence of exchanges that starts with one unit of some currency and ends with **more than one unit of the same currency**, after all fees. You may choose any starting currency. Some exchanges may be unavailable; do not assume that every currency can reach every other currency. Let $m = |E|$.

(a) Assign a real weight to each allowed exchange so that a sequence returning to its starting currency is profitable exactly when its corresponding closed walk has negative total weight. Explain your choice of weights and why an arbitrage opportunity exists if and only if the weighted graph contains a negative-weight cycle.

(b) Use Bellman–Ford to decide whether an arbitrage opportunity exists in $O(n(n + m))$ time. Your algorithm must consider every possible starting currency. Explain why it is correct. You only need to return yes or no, not the exchange sequence. Count each logarithm evaluation as $O(1)$ time.

Solution to Problem 1

(a) A safe edge belongs to the MST.

Let $e = (u, v)$ be the lightest edge crossing a cut $(S, V \setminus S)$. Suppose $e \notin T$. The unique u–v path in T must contain another edge f crossing the cut. Since weights are distinct, $w(e) < w(f)$. Adding e to T creates a cycle containing f; removing f gives a spanning tree of smaller weight. This contradicts the minimality of T, so $e \in T$.

(b) Every MST edge is safe.

Take $e \in T$. Removing e splits T into two components with vertex sets S and $V \setminus S$. If another edge f crossing this cut were lighter than e, then replacing e by f would give a lighter spanning tree. Therefore e is the lightest edge crossing this cut, so it is safe.

Solution to Problem 2

(a) Apply the unsafe-edge test.

Insert e into T, creating a unique cycle. Find this cycle using DFS or BFS, and delete a heaviest edge on it (possibly e itself).

The unsafe-edge test allows us to delete a maximum-weight edge on a cycle while preserving some MST. Since T was an MST, every original non-tree edge h is a maximum-weight edge on the cycle in $T + \{h\}$. Thus, for the correctness argument, we may discard all original non-tree edges using this test, leaving $T + \{e\}$ while preserving some MST of the new graph. Applying the same test to its unique cycle leaves exactly the tree returned by our algorithm, proving optimality.

(b) Reconnect using the safe-edge test.

Use DFS or BFS to identify the two components. Scan the remaining graph edges for a cheapest edge f crossing between them. If none exists, the graph is disconnected; otherwise return $T - \{e\} + \{f\}$.

Each remaining edge of T is still cheapest across the cut obtained by removing it from the original tree. Since this cut crosses no other tree edge, the safe-edge test allows us to retain these edges. Finally, f is cheapest across the cut between the two components, so the same test completes an MST.

Solution to Problem 3

(a) Process components in topological order.

Compute the SCCs and topologically order the *component* graph. Initialize $s.d = 0$ and $v.d = \infty$ for every $v \neq s$. For each component C, in this order:

- Keep the distance labels inherited from processing earlier components.
- Repeat $|C| - 1$ times: relax each edge within C exactly once.
- Then relax each edge from C to another component exactly once.

Return the final distance labels.

(b) Correctness and running time.

With no negative-weight cycles, every reachable vertex has a simple shortest path from s. Such a path visits components in topological order and uses at most $|C| - 1$ internal edges in each component C. These internal edges appear as a subsequence of the $|C| - 1$ passes, taking one successive path edge from each pass. Any edge leaving C is relaxed afterward. Thus the algorithm's relaxation sequence contains every simple shortest path from s as a subsequence. By the path-relaxation lemma, all final distances are correct. Unreachable vertices retain distance ∞.

(Since every component has at most Δ vertices, each internal edge is relaxed at most $\Delta - 1$ times, and each intercomponent edge once. Including the $O(n + m)$ time to compute and order the SCCs, the total running time is $O(n + \Delta m)$.)

Solution to Problem 4

(a) The coupon can change which route is best.

Use four vertices s, a, b, t and exactly these directed edges:

$$w(s,a) = 3, \quad w(a,t) = 4, \quad w(s,b) = 1, \quad w(b,t) = 9.$$

The unique ordinary shortest path is $s \to a \to t$, with cost 7. Making its heaviest edge free leaves cost 3. Instead, take $s \to b \to t$ and make (b,t) free, for cost 1.

(b) Compute the cost before and after the free edge.

Run Dijkstra from s in G to obtain $d_s[u] = \delta(s,u)$. Run Dijkstra from t in G^T (with reversed edge directions), with unchanged edge weights, to obtain $d_t[v] = \delta(v,t)$. Return

$$\min_{(u,v)\in E}\Big(d_s[u] + d_t[v]\Big),$$

considering only edges with finite $d_s[u]$ and $d_t[v]$. If no such edge exists, return ∞.

If a route makes (u,v) free, its remaining cost is at least $d_s[u] + d_t[v]$. Conversely, a shortest route from s to u, followed by the free edge (u,v) and a shortest route from v to t, attains this sum. Thus minimizing over all possible free edges gives the optimum.

(Two Dijkstra runs and one edge scan take $O((n+m)\log n)$ time using a binary heap.)

Solution to Problem 5

(a) Turn multiplication into addition.

The effective exchange rate after fees is $(1-f)r_{ij}$. Give edge (i,j) the weight

$$w(i,j) = -\log\Big((1-f)r_{ij}\Big).$$

For a closed sequence of exchanges C, the final amount from one starting unit is the product of its effective rates. Its total edge weight is

$$\sum_{(i,j)\in C} w(i,j) = -\log\left(\prod_{(i,j)\in C}(1-f)r_{ij}\right).$$

This is negative exactly when the final amount exceeds one. A negative cycle therefore gives an arbitrage opportunity. The converse follows by decomposing a negative closed walk into cycles. Hence profitable exchange sequences exist exactly when the graph has a negative cycle.

(b) Make every currency reachable from one new source.

Build the weighted graph from part (a). Add a new vertex s with an edge of weight 0 from s to every currency. Run Bellman–Ford from s. Return yes if it detects a reachable negative cycle; otherwise return no.

Every original cycle is now reachable from s, regardless of which currencies can reach one another. Adding s creates no new cycles, since no edge enters s. Thus Bellman–Ford detects a negative cycle exactly when one exists among the original currencies, which by part (a) is exactly when arbitrage is possible.